PHYS 142/242 Assignment 1

Wing Huang TSN: 200348262

$$x_{\text{start}} = 0.81$$

Setup

```
import os

for var in ("OMP_NUM_THREADS", "OPENBLAS_NUM_THREADS", "MKL_NUM_THREADS"):
    os.environ.setdefault(var, "4")

import json
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, PillowWriter

os.makedirs("figs", exist_ok=True)

TSN = "200348262"
d = int(TSN[-2:])
s = int(TSN[-3])
x_start = (-1) ** s * (0.5 + d / 200)

T0 = 2 * np.pi
alpha = 2.0
N_D = 600
N_steps = 128
eps = T0 / N_steps


def make_grid(n_d):
    x = np.linspace(-4.0, 4.0, n_d + 1)
    return x, x[1] - x[0]


def initial_psi(x):
    return ((alpha / np.pi) ** 0.25 * np.exp(-alpha / 2 * (x - x_start) ** 2)).astype(np.complex128
    )


def propagator(x, t):
    xb = x[:, None]
    xa = x[None, :]
    return np.sqrt(1 / (2j * np.pi * np.sin(t))) * np.exp(
        1j * ((xa**2 + xb**2) * np.cos(t) - 2 * xa * xb) / (2 * np.sin(t)))


def evolve(psi0, K, dx, n_steps):
```

```
    psi = np.empty((n_steps + 1, psi0.size), dtype=np.complex128)
    psi[0] = psi0
    for n in range(n_steps):
        psi[n + 1] = dx * K @ psi[n]
    return psi


x, dx = make_grid(N_D)
t = np.arange(N_steps + 1) * eps
print("x_start =", x_start)
```

Problem A

$$K_{8\epsilon} = (\Delta x)^7 K_\epsilon^8, \qquad (K_\epsilon)_{ba} = K(x_b, \epsilon; x_a, 0), \qquad \epsilon = T_0/128,\ \Delta x = 8/600.$$

```
[[-0.05955846+0.0178667j  -0.0757611 +0.0217024j  -0.07967385+0.05414053j
  ... -0.46505982-0.30932634j -0.35439299-0.42735575j
  -0.21687407-0.50917202j]
 [-0.0757611 +0.0217024j  -0.0896081 +0.03330227j -0.08308486+0.06724787j
  ... -0.53404108-0.16837676j -0.46141968-0.31131639j
  -0.35439299-0.42735575j]
 [-0.07967385+0.05414053j -0.08308486+0.06724787j -0.06226964+0.09303051j
  ... -0.56407376-0.01040964j -0.53404108-0.16837676j
  -0.46505982-0.30932634j]
 ...
 [-0.46505982-0.30932634j -0.53404108-0.16837676j -0.56407376-0.01040964j
  ... -0.06226964+0.09303051j -0.08308486+0.06724787j
  -0.07967385+0.05414053j]
 [-0.35439299-0.42735575j -0.46141968-0.31131639j -0.53404108-0.16837676j
  ... -0.08308486+0.06724787j -0.0896081 +0.03330227j
  -0.0757611 +0.0217024j ]
 [-0.21687407-0.50917202j -0.35439299-0.42735575j -0.46505982-0.30932634j
  ... -0.07967385+0.05414053j -0.0757611 +0.0217024j
  -0.05955846+0.0178667j ]]
```

```
K_eps = propagator(x, eps)
K_8eps = dx**7 * np.linalg.matrix_power(K_eps, 8)
print(K_8eps)
```

Problem B

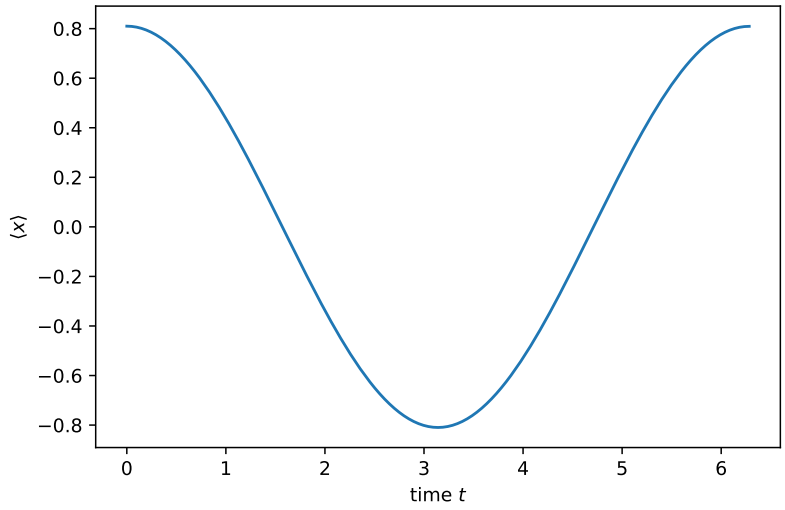


Figure 1: $\langle x \rangle$ versus t over one period $T_0 = 2\pi$.

```
psi = evolve(initial_psi(x), K_eps, dx, N_steps)
P = np.abs(psi) ** 2
x_mean = (x * P).sum(axis=1) * dx

plt.figure(figsize=(6, 4))
plt.plot(t, x_mean)
plt.xlabel("time $t$")
plt.ylabel(r"$\langle x \rangle$")
plt.tight_layout()
plt.savefig("figs/mean_x.pdf")
plt.show()
```

Problem C

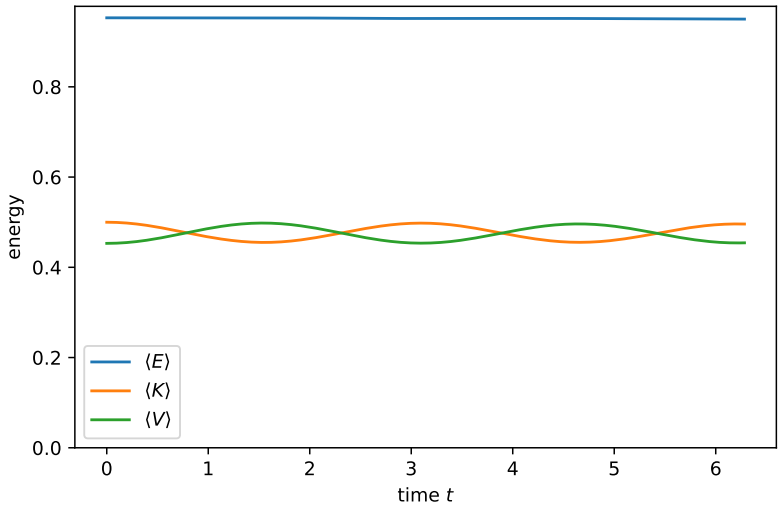


Figure 2: $\langle E \rangle$, $\langle K \rangle$, and $\langle V \rangle$ versus t.

```
V = (0.5 * x**2 * P).sum(axis=1) * dx
K = (0.5 * np.abs(np.gradient(psi, dx, axis=1)) ** 2).sum(axis=1) * dx
E = K + V

plt.figure(figsize=(6, 4))
plt.plot(t, E, label=r"$\langle E \rangle$")
plt.plot(t, K, label=r"$\langle K \rangle$")
plt.plot(t, V, label=r"$\langle V \rangle$")
plt.ylim(bottom=0)
plt.xlabel("time $t$")
plt.ylabel("energy")
plt.legend()
plt.tight_layout()
plt.savefig("figs/energy.pdf")
plt.show()
```

Problem D

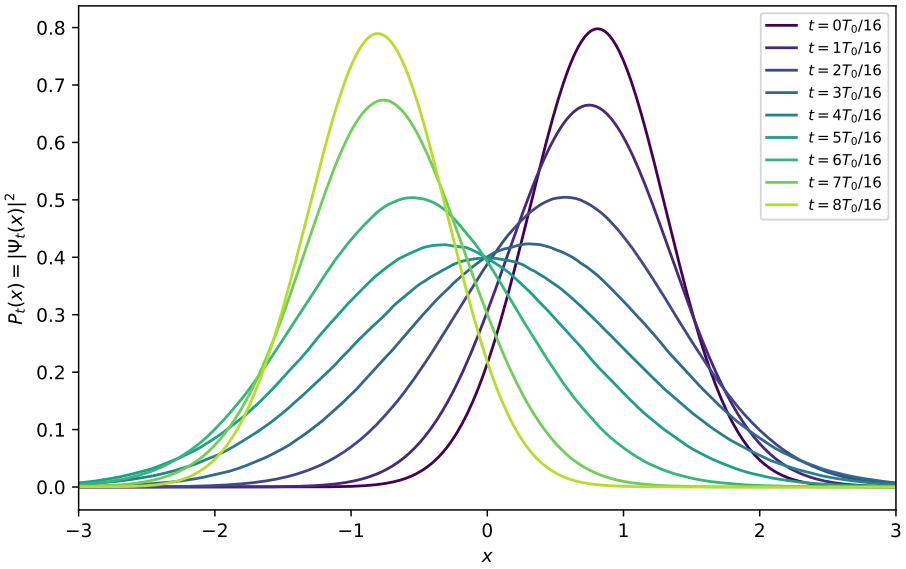


Figure 3: $P_t(x) = |\Psi_t(x)|^2$ at $t = mT_0/16$, $m = 0, \ldots, 8$.

```
P_list = [P[8 * m] for m in range(9)]

plt.figure(figsize=(7, 4.5))
for m, P_t in enumerate(P_list):
    plt.plot(x, P_t, color=plt.cm.viridis(m / 9), label=f"$t = {m}T_0/16$")
plt.xlim(-3, 3)
plt.xlabel("$x$")
plt.ylabel(r"$P_t(x) = |\Psi_t(x)|^2$")
plt.legend(fontsize=8)
plt.tight_layout()
plt.savefig("figs/snapshots.pdf")
plt.show()
```

Problem E

The animation is saved as `animation.gif` in the zip file.

```
fig, ax = plt.subplots(figsize=(6, 4))
line_re, = ax.plot([], [], label=r"Re $\Psi$")
line_im, = ax.plot([], [], label=r"Im $\Psi$")
line_abs, = ax.plot([], [], "k", label=r"$|\Psi|$")
amp = np.abs(psi).max()
ax.set_xlim(-3, 3)
ax.set_ylim(-1.1 * amp, 1.1 * amp)
ax.set_xlabel("$x$")
ax.set_ylabel(r"$\Psi_t(x)$")
ax.legend(loc="upper right")
```

```
def update(n):
    line_re.set_data(x, psi[n].real)
    line_im.set_data(x, psi[n].imag)
    line_abs.set_data(x, np.abs(psi[n]))
    ax.set_title(f"$t = {n}T_0/128$")
    return line_re, line_im, line_abs


FuncAnimation(fig, update, frames=N_steps + 1).save("animation.gif", writer=PillowWriter(fps=20))
plt.close(fig)
```

Problem F

Prediction. As a function of x_a, the phase of $K_\epsilon(x_b, x_a)$ changes at the rate

$$\left|\frac{\partial \phi}{\partial x_a}\right| = \frac{|x_a \cos\epsilon - x_b|}{\sin\epsilon} \lesssim \frac{8}{\sin\epsilon}.$$

The method requires Δx to resolve this oscillation. With $8\Delta x/\sin\epsilon$ as a measure of the phase change per grid step, the three cases give 0.28 for (i), 8.7 for (ii), and 8.7 for (iii). I therefore predict that (ii) and (iii) fail and (i) works.

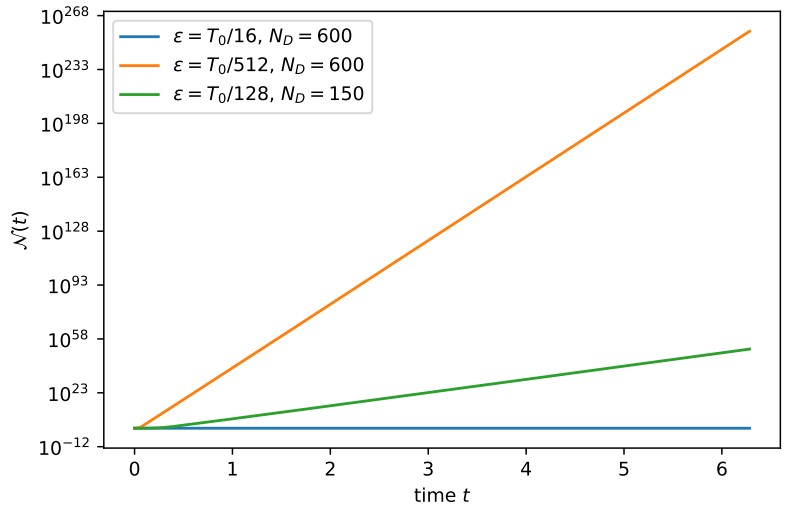


Figure 4: $\mathcal{N}(t) = \Delta x \sum_i |\Psi_t(x_i)|^2$ without renormalization.

Case (i) keeps $\mathcal{N}(T_0) = 0.9995$. Case (ii) grows to $\mathcal{N}(T_0) = 7.4 \times 10^{257}$, and case (iii) to 2.6×10^{51}.

For small ϵ, K_ϵ oscillates faster in x_a, and in the limit $\epsilon \to 0$ it approaches a δ-function. When the phase change per grid step becomes too large, the sum over the grid no longer approximates the integral. $\Delta x\, K_\epsilon$ is then no longer unitary, and the norm grows exponentially with every step.

By Poisson summation, the grid sum fails once $|\partial\phi/\partial x_a|$ reaches $2\pi/\Delta x$ somewhere on the grid. The condition for the method to work is $8\Delta x/\sin\epsilon < 2\pi$, i.e.

$$\epsilon_{\min} \approx \frac{4\Delta x}{\pi} = \frac{32}{\pi N_D}.$$

For $N_D = 600$ this gives $\epsilon_{\min} \approx 0.017 \approx T_0/370$; numerically, the first failure is at $\epsilon = T_0/372$. Doubling N_D halves $\epsilon_{\min}$: the estimate is $T_0/740$ for $N_D = 1200$, and the first numerical failure is at $T_0/744$.

```
def norm_vs_t(eps_c, n_d):
    xc, dxc = make_grid(n_d)
    n_steps = int(round(T0 / eps_c))
    psi_c = evolve(initial_psi(xc), propagator(xc, eps_c), dxc, n_steps)
    return np.arange(n_steps + 1) * eps_c, (np.abs(psi_c) ** 2).sum(axis=1) * dxc


plt.figure(figsize=(6, 4))
for n, n_d in ((16, 600), (512, 600), (128, 150)):
    t_c, N_c = norm_vs_t(T0 / n, n_d)
    plt.semilogy(t_c, N_c, label=rf"$\epsilon = T_0/{n}$, $N_D = {n_d}$")
    print(f"eps = T0/{n}, N_D = {n_d}: N(T0) = {N_c[-1]:.3e}")
plt.xlabel("time $t$")
plt.ylabel(r"$\mathcal{N}(t)$")
plt.legend()
plt.tight_layout()
plt.savefig("figs/norm.pdf")
plt.show()

for n_d in (600, 1200):
    for n in range(100, 1000, 4):
        if abs(norm_vs_t(T0 / n, n_d)[1][-1] - 1) > 1e-2:
            print(f"N_D = {n_d}: fails first at eps = T0/{n} = {T0 / n:.4f}")
            break
```

AI use appendix

I used an AI assistant (Claude, Anthropic) only for debugging. When my code raised errors or gave unexpected output, I showed the relevant code and error messages to the assistant and asked it to help locate the cause. I then checked each suggested fix myself before applying it.